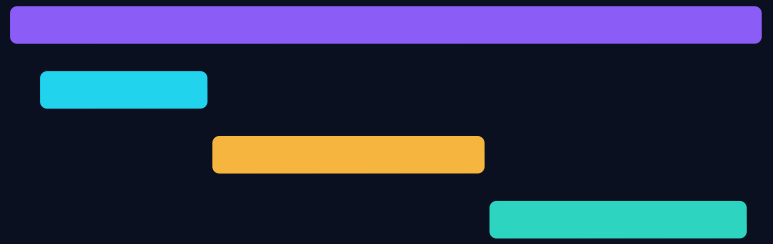
DATA SENSE · AI DISCUSSIONS

AI Observability

Foundations

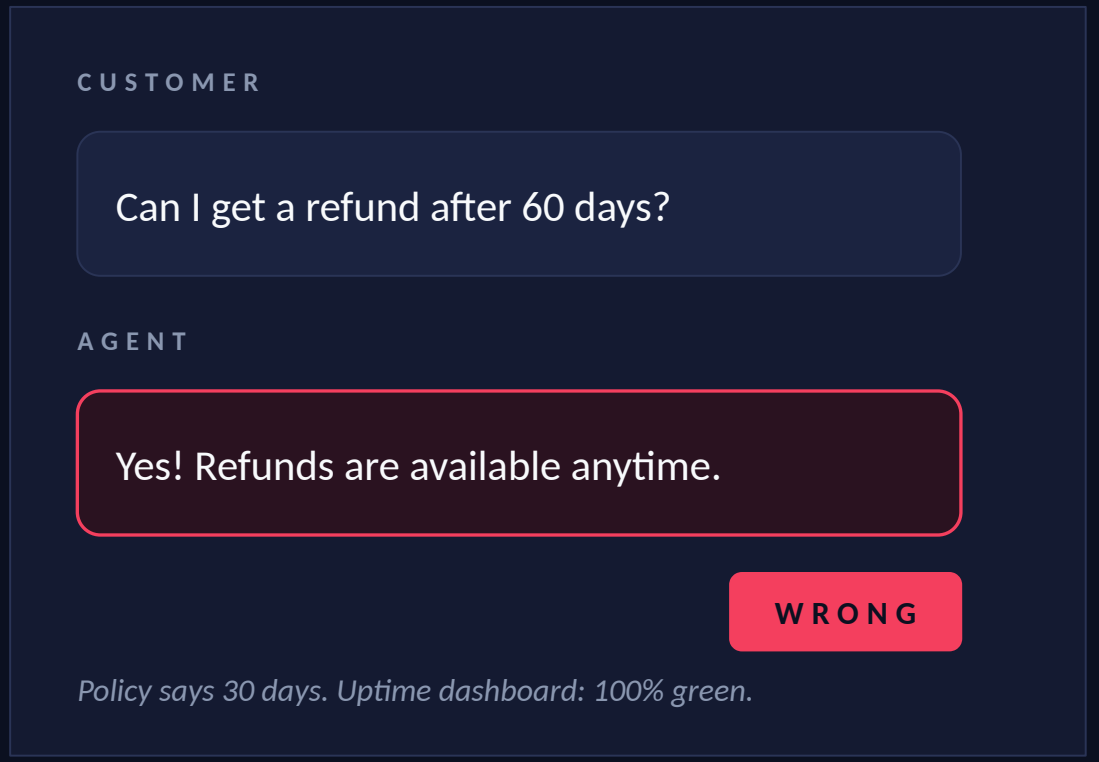


See inside your LLM apps and agents: every step, every token, every decision.

Tuesday, October 20, 2026 · 7:00 PM PT

START WITH A FAILURE

Your agent just gave a wrong answer. Why?



- Which prompt was sent?
- Which tool returned what?
- Which model, which version?
- How long did each step take?
- What did it cost?

TONIGHT

Six parts, one running example

01
Why AI needs observability
Non-determinism, agents, cost, silent failures

02
The four signals
Logs, metrics, traces and spans on one function

03
OpenTelemetry
The open standard, the pipeline, GenAI conventions

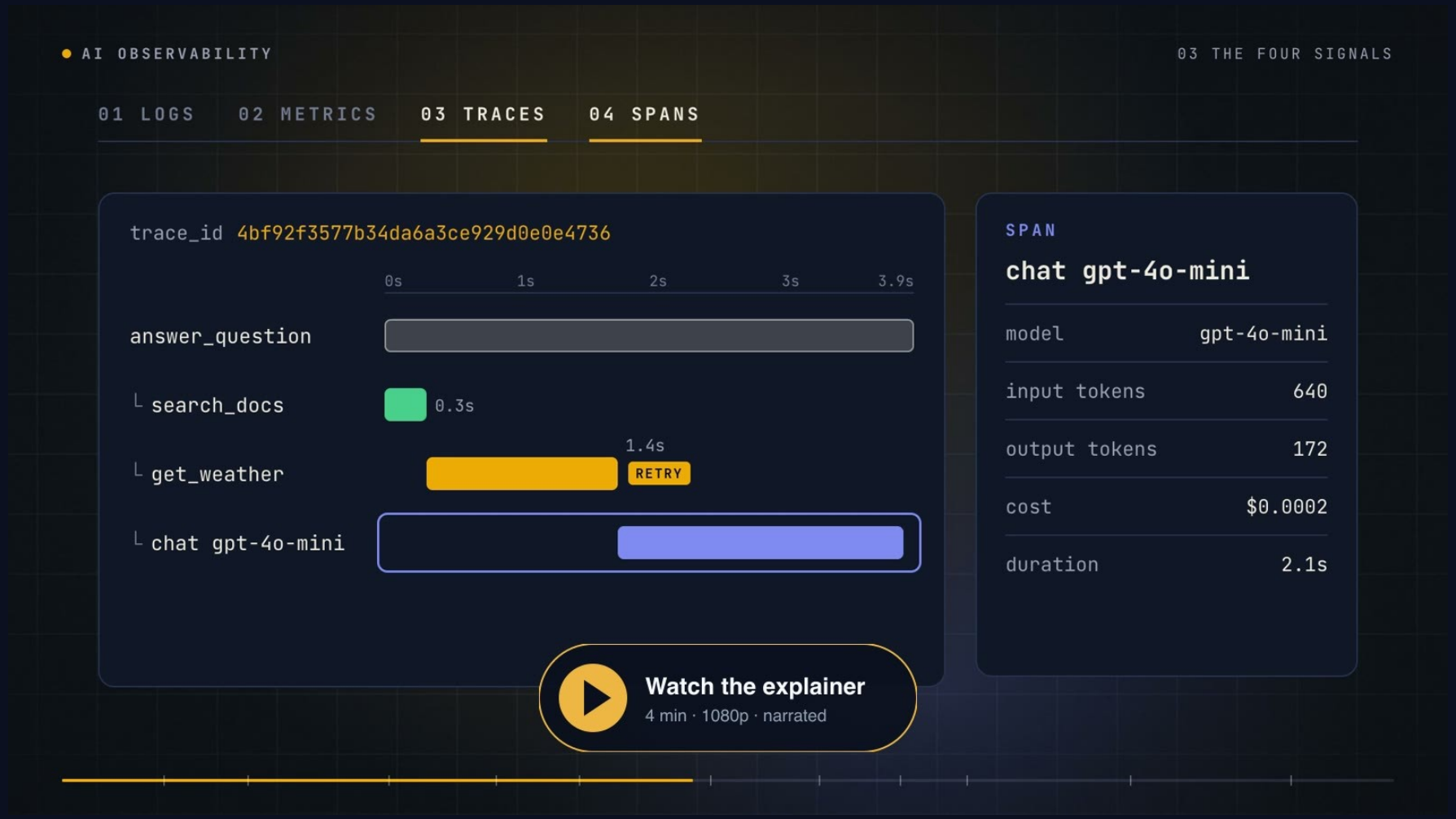
04
What to measure
Latency, errors, tokens, cost, quality

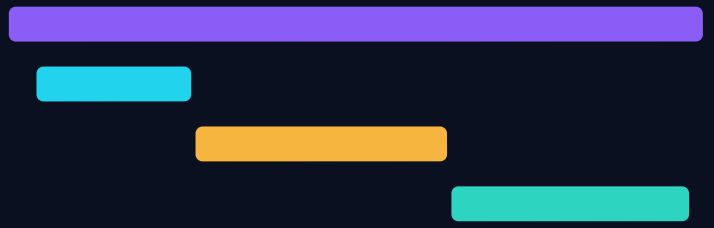
05
Hands-on
Langfuse on localhost, LangSmith, plain OTel

06
The landscape
Datadog, Dynatrace and how to choose

FOUR MINUTES

AI observability, explained





01

Why AI needs observability

Monitoring tells you that. Observability tells you why.

TWO DIFFERENT QUESTIONS

Monitoring vs observability

MONITORING

Is it broken?

- Known questions, decided in advance
- Dashboards and threshold alerts
- Great for failures you predicted

OBSERVABILITY

Why is it broken?

- New questions, asked after the fact
- Answered from data the system emits
- Built for failures you did not predict

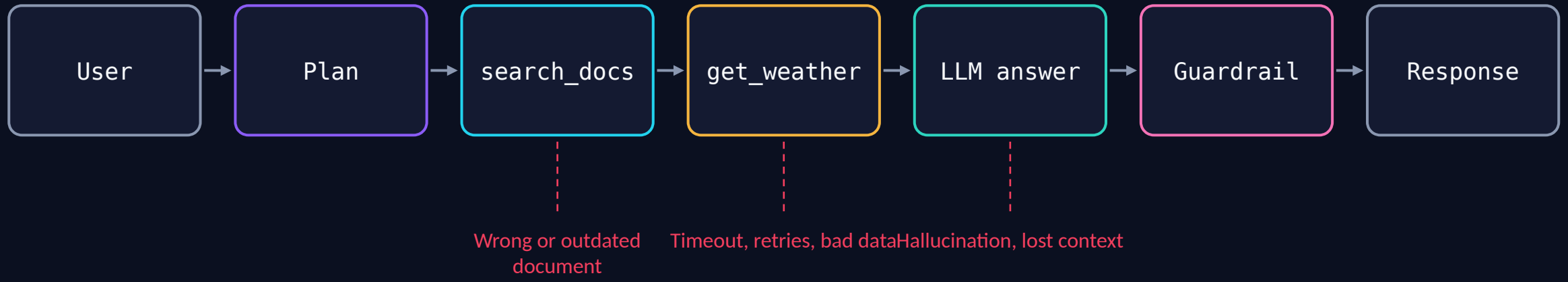
THE RULES CHANGED

Traditional apps vs AI apps

TRADITIONAL APP	AI APP
Same input, same output	**Same input, different output**
Failure is an exception or a 500	**Failure is a confident wrong answer with HTTP 200**
Cost is roughly fixed infrastructure	**Every request costs money, measured in tokens**
Logic lives in code	**Logic lives in prompts, models and tools**
Milliseconds, predictable	**Seconds, highly variable**

An agent is a chain of decisions

Five to twenty steps per request. Any one of them can ruin the final answer.



SIX QUESTIONS

What AI observability answers

Debugging

Which step failed, and with what input?

Performance

Where did the eight seconds go?

Cost

Which feature, user or model burns the budget?

Quality

Did answers get worse after the prompt change?

Safety

PII leaks, prompt injection, policy violations

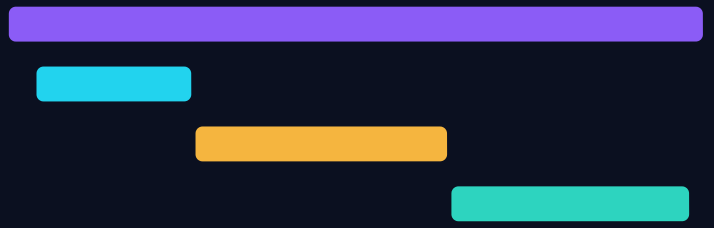
Product

What are users actually asking for?

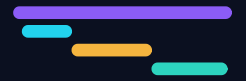
02

The four signals

Logs, metrics, traces and spans, on one function.



Four signals, four questions





Logs

What happened?

Timestamped events with rich detail



Metrics

How much, how often?

Numbers aggregated over time



Traces

Where, and why?

The full journey of one request



Spans

Which step?

One timed operation inside a trace

THE RUNNING EXAMPLE

One tiny weather assistant

```python
def get_weather(city):
    return FAKE_WEATHER[city]          # step 1

def ask_ai(question):
    return call gpt-4o-mini            # step 2

def weather_assistant(city):
    weather = get_weather(city)
    return ask_ai(f"Weather is {weather}. Umbrella?")
```

“It’s rainy in Paris. Should I carry an umbrella?”

`get_weather` looks up the weather

`ask_ai` asks the AI model, costs tokens

Two steps, plain Python. This exact function is in notebook 01, and we trace it in Langfuse and LangSmith later.

SIGNAL 1

Logs: what happened

```
19:02:11.746  INFO  request received  city="London"
19:02:11.747  INFO  request received  city="Tokyo"
19:02:11.747  INFO  request received  city="Dubai"
19:02:11.874  INFO  docs retrieved  count=1
19:02:11.880  INFO  docs retrieved  count=1
19:02:12.931  ERROR weather failed  timeout
19:02:13.402  INFO  llm answered  tokens=97
19:02:13.655  INFO  llm answered  tokens=104
```

GOOD AT

Rich detail: error messages, parameters, audit trails.

BLIND SPOT

Three users at once. Which “llm answered” belongs to London? Which request did the error break?

Metrics: how much, how often

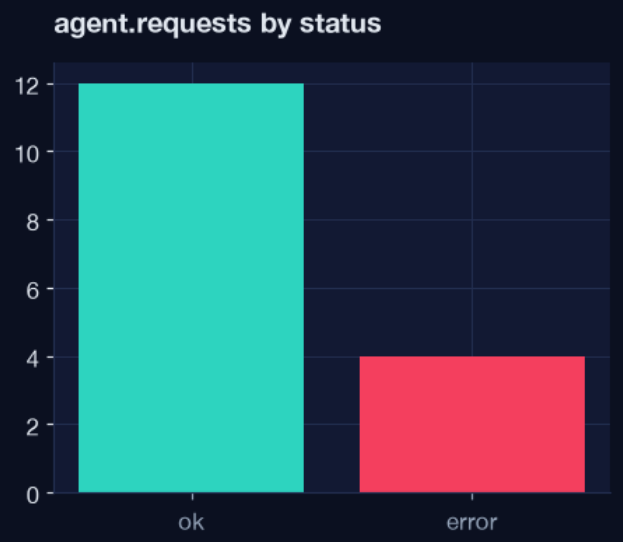
1.18s
p50 latency

2.74s
p95 latency

25%
requests failed

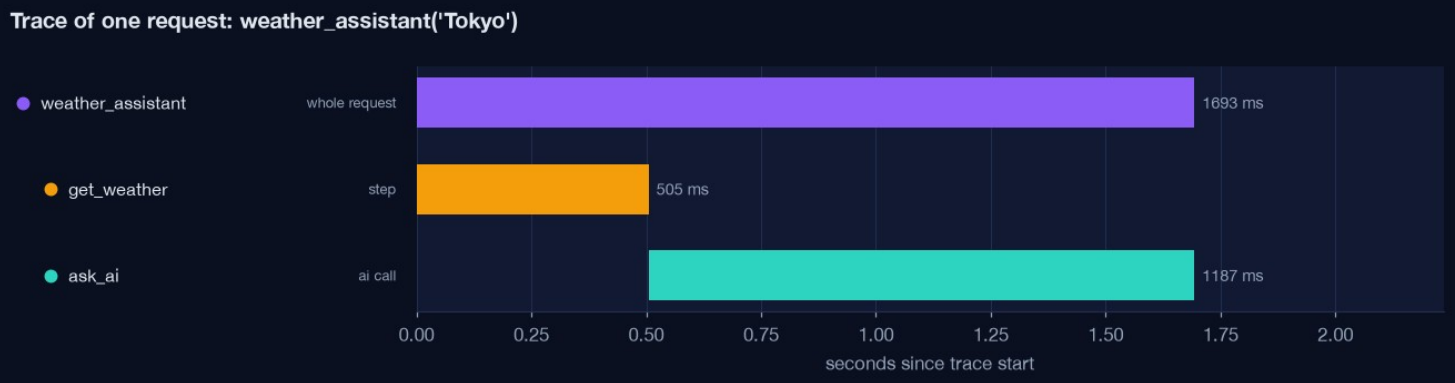
16
requests





Cheap, aggregated, perfect for dashboards and alerts. But it tells you p95 got worse, not which request or why.

Traces and spans: where, and why



A real trace of weather_assistant("Tokyo") from notebook 01. Time runs left to right.

Trace

One request end to end, one trace_id

Span

One step: name, start, end, parent, attributes, status

Nesting

Child spans sit inside their parent, so the tree draws itself

UNDER THE HOOD

Anatomy of a span


```
{
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "parent_span_id": "a3ce929d0e0e4736",
  "name": "chat gpt-4o-mini",
  "start / end": "19:02:13.100" / "19:02:15.200",
  "status": "OK",
  "attributes": {
    "gen_ai.operation.name": "chat",
    "gen_ai.request.model": "gpt-4o-mini",
    "gen_ai.usage.input_tokens": 640,
    "gen_ai.usage.output_tokens": 172
  }
}
```


IDENTITY

`trace_id + span_id + parent`

How four flat records become one tree

TIMING

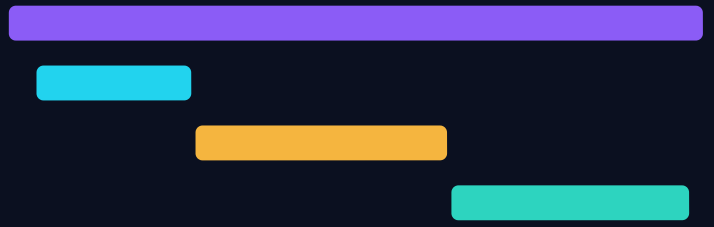
`start, end, status`

Latency and errors per step

ATTRIBUTES

`gen_ai.*`

Model, tokens, and from them, cost



03

OpenTelemetry

Instrument once, export anywhere.

THE OPEN STANDARD

What is OpenTelemetry?

An open source, vendor-neutral CNCF project, supported by every major observability vendor.



API + SDK

Create spans, metrics and logs in your code

OTLP

The wire protocol that ships them, over HTTP or gRPC

Collector

Optional process that filters, redacts, samples and routes

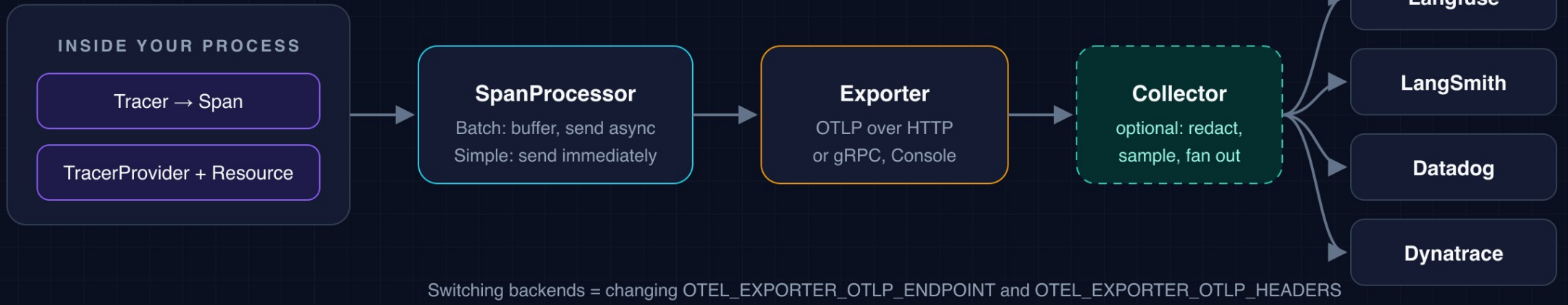
Semantic conventions

Agreed names, such as gen_ai.request.model

From your process to any backend

HOW A SPAN TRAVELS

Instrument once, export anywhere



Switching backends = changing OTEL_EXPORTER_OTLP_ENDPOINT and OTEL_EXPORTER_OTLP_HEADERS

IN CODE

Creating spans with OpenTelemetry

```python
def weather_assistant(city):
    with tracer.start_as_current_span("weather_assistant"):

        with tracer.start_as_current_span("get_weather"):
            weather = get_weather(city)

        with tracer.start_as_current_span("ask_ai"):
            return ask_ai(...)
```

`with ... :`

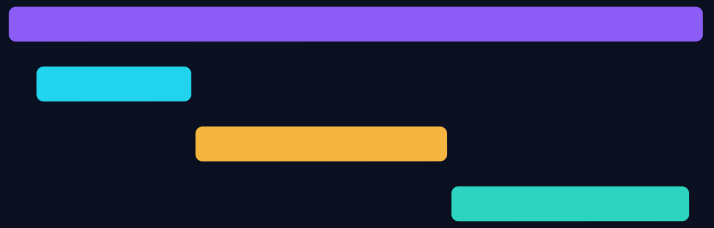
everything inside the block is timed and recorded as one span

`nested with`

a span started inside another becomes its child, so you get a tree

`one line per step`

no other changes to your function



04

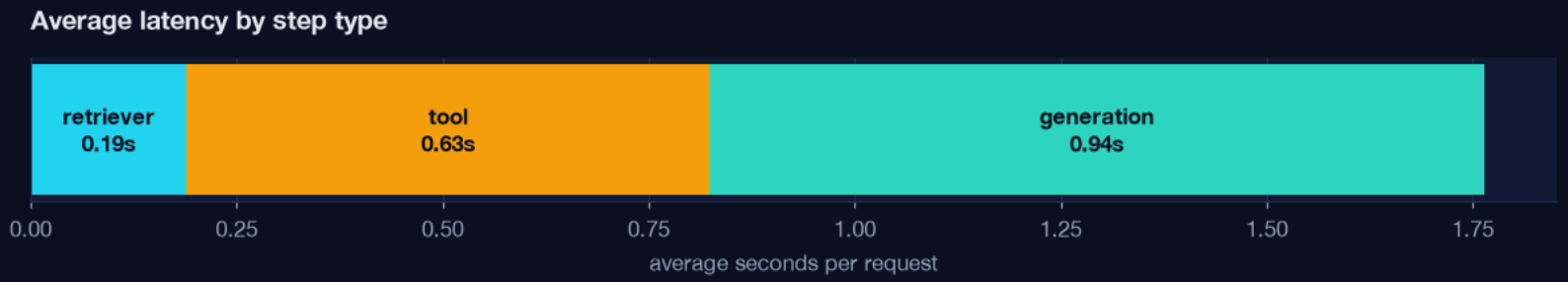
What to measure

Latency, errors, tokens, cost and quality.

Watch the tail, not the average







p50 / p95 / p99

end-to-end latency percentiles

Time to first token

what users feel when streaming

Per-step latency

retrieval vs tools vs LLM

Errors

429s, timeouts, tool and parse failures, refusals

A one-line prompt change doubled the bill



31
avg output tokens, concise prompt

87
avg output tokens, friendly prompt

2.1×
monthly cost, same model

Cost = tokens × model price, per span, rolled up per trace, user and feature. Watch for runaway agent loops, bloated context windows and retries multiplying spend.

QUALITY AND CONTEXT

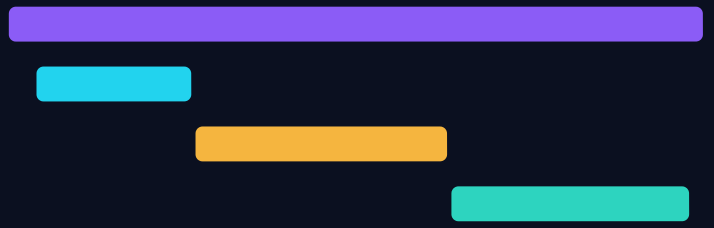
For whom, and how good?

Scores and feedback	Thumbs up or down, LLM-as-a-judge, human labels
Sessions	Multi-turn conversations grouped together
Users	Per-user usage, cost and issues
Metadata and tags	Prompt version, release, environment, feature flag
Guardrails	PII detected, injection attempts, policy blocks

05

Hands-on

Langfuse on localhost, LangSmith, and plain OpenTelemetry.



FOLLOW ALONG

The AI Observability Lab



github.com/fnusatvik07/ai-observability-lab

```
make setup
make up
make lab
```

00 Setup: start the local stack

01 What is observability?

02 Your first trace in Langfuse

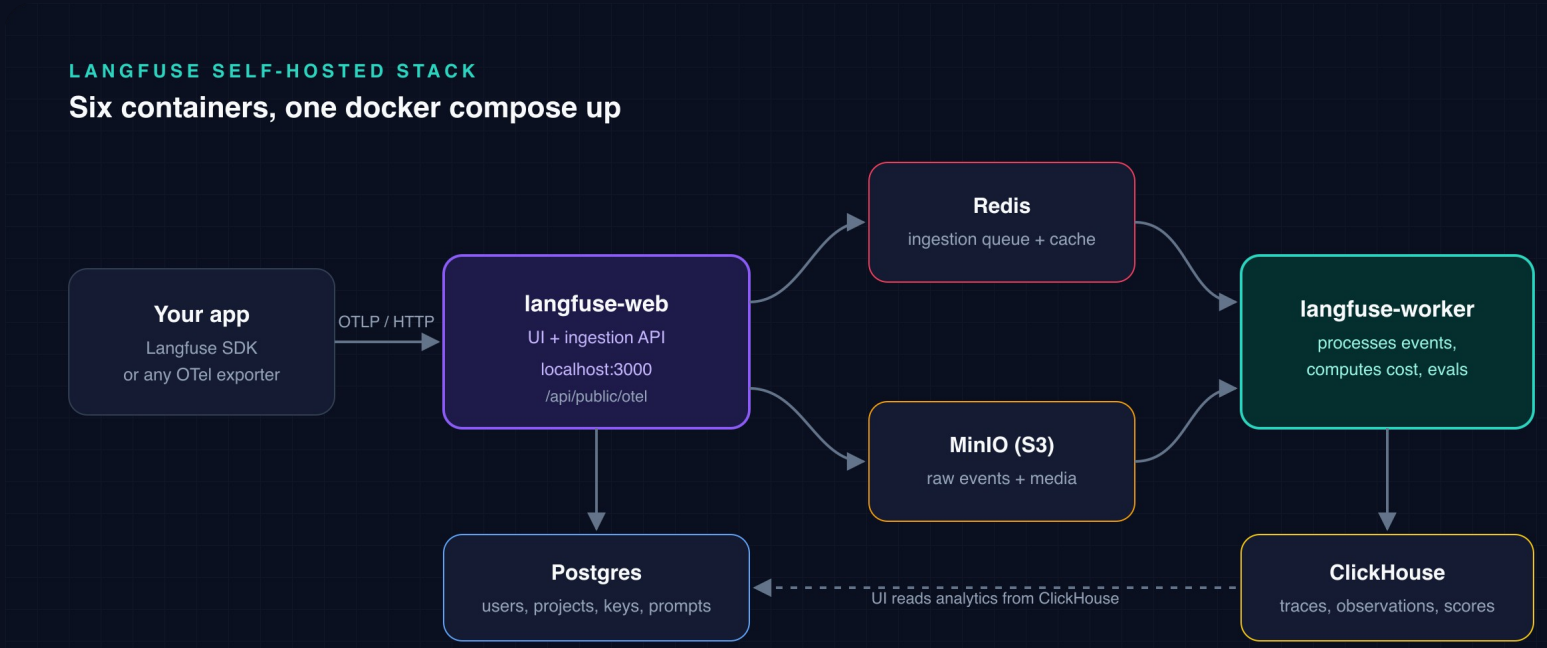
03 The same function in LangSmith

04 Beyond Langfuse and LangSmith

\+ advanced/: deeper notebooks for later

DEMO 1

Langfuse on localhost




```
git clone langfuse
docker compose up -d
open localhost:3000
```

Headless init creates the user, project and API keys on first boot. No sign-up form.

DEMO 2

Add Langfuse in two changes

```python
from langfuse import observe
from langfuse.openai import OpenAI      # NEW 1

@observe()                              # NEW 2
def get_weather(city): ...

@observe()
def ask_ai(question): ...

@observe()
def weather_assistant(city): ...
```

NEW 1

Import OpenAI from langfuse.openai. Every AI call is recorded: prompt, answer, model, tokens, cost.

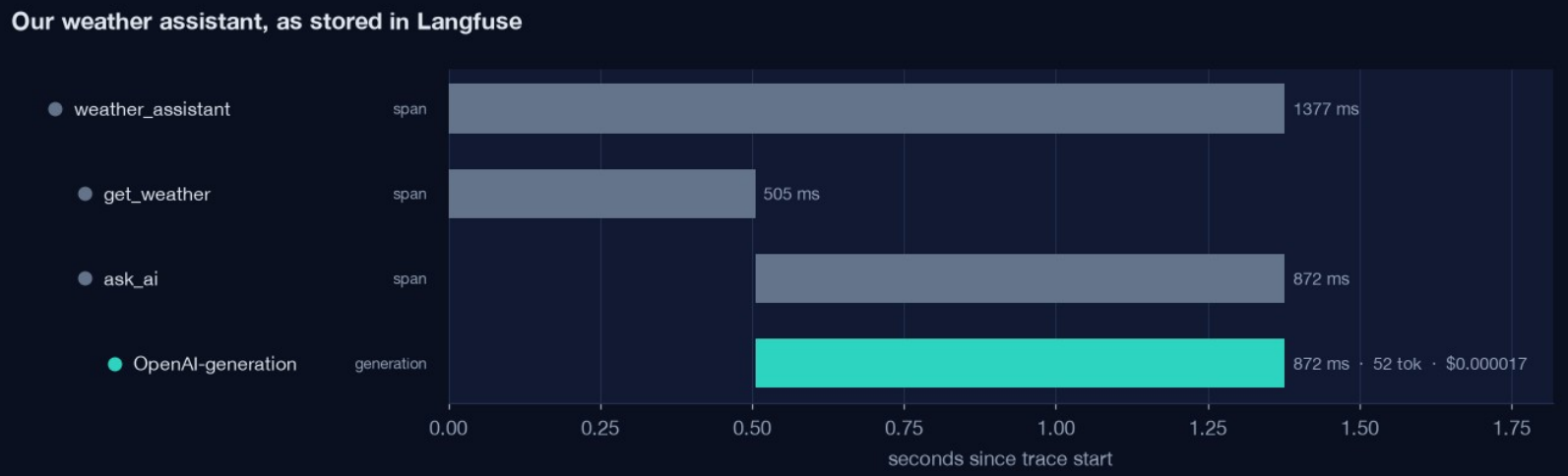
NEW 2

@observe() turns each function into a span. Nested calls become child spans.

then

langfuse.flush() sends everything. Open localhost:3000, click Tracing.

What you see in Langfuse



The same weather_assistant trace, fetched from Langfuse. The last row was added for us by the OpenAI import.

`weather_assistant`

input, final answer, total time

`get_weather`

about 0.5 s

`OpenAI-generation`

prompt, reply, model, tokens, cost

`Users page`

who asked (optional step)

DEMO 3

Same agent, in LangSmith



```python
from langsmith import traceable
from langsmith.wrappers import wrap_openai

client = wrap_openai(OpenAI())

@traceable
def get_weather(city): ...
```

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_...
```

LangSmith	Langfuse	OpenTelemetry
Project	Project	service.name
Trace	Trace	Trace
Run	Observation	Span
Run type	Observation type	operation.name
Thread	Session	
Feedback	Score	

Same ideas, different names. Learn the concept once.

DEMO 4

Other tools: change the address, not the code

Langfuse

```
endpoint http://localhost:3000/api/public/otel
headers  Authorization: Basic <your keys>
```

LangSmith

```
endpoint https://api.smith.langchain.com/otel
headers  x-api-key: <your key>
```

Dynatrace

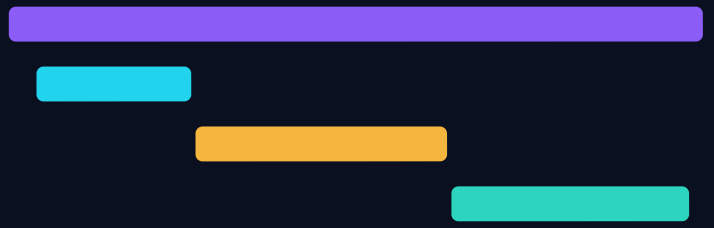
```
endpoint https://<id>.live.dynatrace.com/api/v2/otlp
headers  Authorization: Bearer <token>
```

Your functions and your with blocks stay exactly the same. Notebook 04 sends plain OpenTelemetry to Langfuse with zero Langfuse code.

06

The landscape

Different names, same ideas.



WHO DOES WHAT

The tool landscape

LLM-NATIVE

Built for prompts, agents and evals

LangSmith

Managed, LangChain depth

Langfuse

Open source, self-host

Arize Phoenix

Open source, OpenInference

APM AND ENTERPRISE

AI next to your infrastructure

Datadog

LLM Observability

Dynatrace

Full-stack, AI root cause

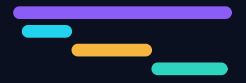
Grafana

Tempo, open source

Underneath, increasingly: OpenTelemetry

ENTERPRISE

Datadog and Dynatrace





Datadog LLM Observability

- Span kinds: llm, workflow, agent, tool, task, embedding, retrieval
- Cost and latency dashboards, evaluations
- PII and prompt-injection scanning
- Native OTel GenAI ingest, v1.37+

```
DD_LLMOBS_ENABLED=1 ddtrace-run python app.py
```



Dynatrace

- AI observability on top of full-stack APM
- Ingests OTLP and OpenLLMetry traces
- Links LLM spans to services, infra and business metrics
- Davis AI for root cause analysis

```
endpoint: .../api/v2/otlp
Authorization: Bearer <token>
```

DECISION GUIDE

How to choose

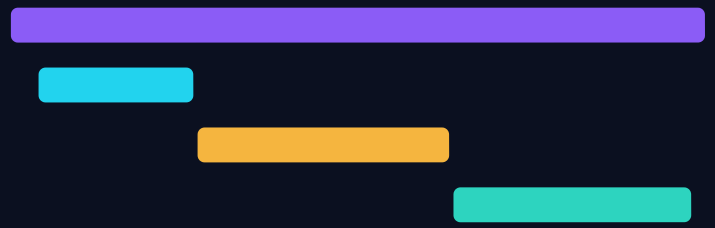
Data must stay in-house?	→	Self-host Langfuse or Phoenix
LangChain or LangGraph everywhere?	→	LangSmith
Company already on Datadog or Dynatrace?	→	Their LLM module, fed by OpenTelemetry
Not sure yet?	→	Instrument with OTel, try two backends, keep the one your team opens daily

RECAP

Five things to remember

- **AI fails silently.** Observe every step, not just uptime.
- **Four signals.** Logs are events, metrics are numbers, traces are journeys, spans are steps.
- **OpenTelemetry.** Instrument once, export anywhere. GenAI conventions name the fields.
- **Measure what matters.** Latency, errors, tokens, cost, quality. Tag user, session and version.
- **Tools differ in names,** not in concepts.

Questions?



COMING NEXT

Evaluations

LLM-as-a-judge, datasets, experiments

Prompt management

Versioning and linking prompts to traces

Alerts and dashboards

SLOs on latency, cost and quality

Production pipelines

Collector, sampling, redaction at scale

github.com/fnusatvik07/ai-observability-lab